

Instituto Tecnológico Superior de Lerdo

Ingeniería Informática

Programación Orientada a Objetos

Portafolio de poo



Alumno:

Francisco Javier Aguilar mejía

Docente:

Jesús Salas Marín

Fecha:

1 de junio de 2026

Índice

Índice	2
Introducción	4
Propósito del portafolio	4
Competencias desarrolladas	4
Importancia de la Programación Orientada a Objetos	4
Importancia de GitHub y el trabajo organizado	4
Proyecto 1: Examen práctico contra reloj – Corte 1	5
Objetivo del proyecto	5
Problema planteado	5
Conceptos de POO aplicados	5
Explicación del funcionamiento	5
Capturas del proyecto	6
Dificultades encontradas	6
Soluciones aplicadas	6
Proyecto 2: Analizador de Consumo Energético en Servidores	7
Objetivo del proyecto	7
Problema planteado	7
Conceptos de POO aplicados	7
Explicación del funcionamiento	7
Capturas del proyecto	8
Dificultades encontradas	8
Soluciones aplicadas	9
Proyecto 3: Gestión de Inventario – Arreglos Unidimensionales en PHP	10
Objetivo del proyecto	10
Problema planteado	10
Conceptos de POO aplicados	10
Explicación del funcionamiento	10
Capturas del proyecto	11
Dificultades encontradas	11
Soluciones aplicadas	12
Proyecto 4: Gestión de Bitácoras en Archivos de Texto (PHP)	13
Objetivo del proyecto	13
Problema planteado	13
Conceptos de POO aplicados	13
Explicación del funcionamiento	13

Capturas del proyecto	14
Dificultades encontradas	14
Soluciones aplicadas.....	14
Proyecto 5: Sistema de Usuarios con Herencia en Python	15
Objetivo del proyecto.....	15
Problema planteado	15
Conceptos de POO aplicados	15
Explicación del funcionamiento	15
Capturas del proyecto	16
Dificultades encontradas	19
Soluciones aplicadas.....	19
Reflexión Final	20
¿Cuál proyecto fue más difícil?	20
¿Cuál proyecto fue más útil?	20
¿Qué habilidad mejoró más?.....	20
¿Qué aprendiste sobre Programación Orientada a Objetos?	20
¿Qué aprendiste sobre trabajo organizado?.....	20
Conclusiones	21
Crecimiento técnico	21
Importancia del desarrollo de proyectos	21
Importancia de la documentación y GitHub	21

Introducción

Propósito del portafolio

Este portafolio tiene como propósito documentar de manera estructurada y reflexiva los proyectos desarrollados a lo largo del semestre en la materia de Programación Orientada a Objetos. Cada proyecto representa un hito en el aprendizaje progresivo de conceptos fundamentales del paradigma OOP, y su recopilación evidencia el crecimiento técnico, la capacidad de resolución de problemas y la madurez en el uso de herramientas de desarrollo profesional.

Competencias desarrolladas

- Diseño e implementación de clases con encapsulamiento, herencia y polimorfismo.
- Uso de namespaces y organización modular de proyectos en PHP.
- Manejo de excepciones y validación de datos en tiempo de ejecución.
- Desarrollo de interfaces web con PHP, HTML y CSS.
- Manipulación de archivos de texto como mecanismo de persistencia ligera.
- Implementación de algoritmos numéricos (integración trapezoidal).
- Modelado de sistemas con herencia múltiple en Python.
- Trabajo con arreglos unidimensionales y funciones de procesamiento de datos.

Importancia de la Programación Orientada a Objetos

La Programación Orientada a Objetos (POO) es el paradigma dominante en el desarrollo de software moderno. A través de sus pilares —encapsulamiento, herencia, polimorfismo y abstracción— permite construir sistemas complejos de manera organizada, reutilizable y mantenible. En el contexto profesional, la POO es la base de los grandes frameworks, sistemas empresariales y plataformas en la nube. Dominar este paradigma es, en esencia, hablar el lenguaje del software de producción.

Importancia de GitHub y el trabajo organizado

GitHub y el control de versiones con Git son herramientas indispensables en el entorno laboral tecnológico actual. Permiten llevar un historial detallado de los cambios en el código, colaborar en equipo sin conflictos, revertir errores y demostrar públicamente las habilidades técnicas de un desarrollador. Un repositorio bien organizado es la primera carta de presentación ante un empleador. Este portafolio también busca reflejar esa cultura de trabajo ordenado y profesional.

Proyecto 1: Examen práctico contra reloj – Corte 1

Proyecto	Examen práctico contra reloj – Corte 1
Materia	Programación Orientada a Objetos
Lenguaje	PHP
Tipo de actividad	Examen práctico (30 minutos)
Corte	Corte 1

Objetivo del proyecto

Medir la precisión técnica en la sintaxis de PHP y la capacidad de modelado de datos mediante la definición de clases con propiedades tipadas, sin el uso de métodos, bajo condiciones de tiempo limitado.

Problema planteado

Se requería modelar dos entidades del mundo real: un paquete logístico para la empresa FastDelivery y un sensor para monitoreo de plantas. Para cada entidad debía crearse una clase PHP con propiedades correctamente tipadas, incluyendo tipos primitivos (string, float, bool, int) y objetos predefinidos de PHP (DateTime). Adicionalmente, se debía demostrar el concepto de encapsulamiento mediante una propiedad privada y documentar en código qué ocurre al intentar acceder a ella desde fuera de la clase.

Conceptos de POO aplicados

- Clases y propiedades tipadas en PHP (strict typing).
- Modificadores de acceso: public y private.
- Encapsulamiento: restricción de acceso a propiedades internas.
- Instanciación de objetos y uso de propiedades públicas.
- Uso de la clase predefinida DateTime de PHP.
- Organización de archivos en carpetas (src/Logistica/).
- Uso de require_once para importar clases externas.

Explicación del funcionamiento

El proyecto consistió en tres ejercicios cronometrados ejecutados en 30 minutos:

- Ejercicio 1 – Clase Paquete: Se creó el archivo src/Logistica/Paquete.php con la clase Paquete, que define las propiedades públicas codigoSeguimiento (string), pesoKilogramos (float) y esFragil (bool), más la propiedad privada costoInterno (float).
- Ejercicio 2 – Clase Sensor: Se creó Sensor.php con las propiedades id (int), marca (string), ultimaLectura (DateTime) y rangoMaximo (float).
- Ejercicio 3 – index.php: Se importaron ambas clases, se instanciaron dos objetos Paquete y uno Sensor, se asignaron valores a las propiedades públicas y se comentó el

intento de acceso a costoInterno, explicando que PHP lanza un error fatal de acceso a miembro privado.

Capturas del proyecto

```
=== Paquete A === Código: FD-2024-001 Peso: 3.75 kg Frágil: Sí === Sensor === ID: 1 Marca: Bosch Última lectura: 2026-06-02 00:50:02 Rango máximo: 100
```

Dificultades encontradas

- La presión del tiempo dificultó recordar la sintaxis exacta del tipado de propiedades en PHP 8.
- Definir correctamente el tipo DateTime para la propiedad ultimaLectura generó dudas iniciales.
- Organizar los archivos en subcarpetas y usar rutas correctas en require_once tomó tiempo bajo presión.

Soluciones aplicadas

- Repasar la sintaxis básica de declaración de propiedades tipadas antes del examen.
- Verificar que el tipo DateTime se escriba con mayúscula inicial, ya que es una clase de PHP.
- Practicar previamente la estructura de carpetas del proyecto para automatizar esa parte.

Proyecto 2: Analizador de Consumo Energético en Servidores

Proyecto	Analizador de Consumo Energético en Servidores
Materia	Programación Orientada a Objetos
Lenguaje	PHP
Tipo de actividad	ABPj – Actividad de evaluación Corte 2
Unidad	Unidad 3 – Métodos

Objetivo del proyecto

Desarrollar una herramienta web profesional que permita a los administradores de sistemas calcular la Energía Total Consumida (en Joules) por un servidor en un intervalo de tiempo, aplicando integración numérica mediante la Regla del Trapecio, encapsulamiento y manejo de excepciones en PHP.

Problema planteado

En un centro de datos, el consumo energético de un servidor varía según la carga de la CPU. Para facturar a los clientes de la nube o calcular la huella de carbono, es necesario calcular la integral definida de la función de potencia $P(t)$ en el intervalo $[a, b]$. Se requería completar el código de una clase con espacios en blanco, corregir errores y construir la interfaz web completa.

Conceptos de POO aplicados

- Namespaces en PHP (App\Calculo) para organización profesional del código.
- Encapsulamiento: propiedades private (\$inicio, \$fin, \$pasos).
- Manejo de Excepciones con try-catch y throw new InvalidArgumentException.
- Métodos: constructor tipado, método privado (funcionPotencia), método público (calcularEnergiaTotal).
- Integración numérica – Regla del Trapecio compuesta.
- Arquitectura MVC ligera: separación entre lógica (IntegradorNumerico.php) e interfaz (index.php).
- Uso de require_once y use para importar namespaces.

Explicación del funcionamiento

El proyecto se estructuró en dos archivos principales:

- IntegradorNumerico.php: Clase con namespace App\Calculo. El constructor valida que $a < b$ y que $n > 0$, lanzando excepciones tipadas. El método privado funcionPotencia(t) modela $P(t) = t^2 + 2t$. El método público calcularEnergiaTotal() implementa la Regla del Trapecio: $h = (b-a)/n$, luego acumula $f(a)/2 + f(b)/2 +$ suma de $f(t_i)$ para $i=1..n-1$, multiplicando por h .

- index.php: Formulario HTML que captura `t_inicio`, `t_fin` y `precision`. Al recibir el POST, instancia `IntegradorNumerico` con los valores casteados a `float` e `int`, invoca `calcularEnergiaTotal()` y muestra el resultado formateado. Los errores de validación se muestran en un bloque HTML de error.

Capturas del proyecto

Monitor de Energía (DataCenter)

Tiempo Inicial (s):

Tiempo Final (s):

Precisión (n subintervalos):

Perfil de Consumo:

Unidad:

Calcular

Tabla de Ejemplo

n (subintervalos)	Resultado Aproximado	Error Absoluto
10	435.0000	1.6700
100	433.3500	0.0200
1000	433.3335	0.0035

Consumo Total: 26,468,935.1080 Joules

Cálculo basado en la integral definida de la carga del servidor.

Error: 0.0034999999998604

Dificultades encontradas

- Completar correctamente los modificadores de acceso (`private`) de las propiedades sin ver la documentación.
- Entender la sintaxis de `throw new InvalidArgumentException` con el namespace correcto.
- Implementar la Regla del Trapecio correctamente, recordando dividir el primer y último término entre 2.
- Conectar el namespace con el `use` en `index.php` sin errores de carga.

Soluciones aplicadas

- Revisar la teoría de modificadores de acceso de PHP y practicar con ejemplos cortos.
- Usar `InvalidArgumentException` como clase de excepción estándar, que no requiere namespace adicional.
- Derivar la fórmula del trapecio paso a paso antes de codificarla para evitar errores lógicos.
- Probar `require_once` con rutas absolutas primero y luego ajustar a rutas relativas.

Proyecto 3: Gestión de Inventario – Arreglos Unidimensionales en PHP

Proyecto	Gestión de Inventario – Arreglos Unidimensionales
Materia	Programación Orientada a Objetos
Lenguaje	PHP
Tipo de actividad	Actividad Integradora
Archivos	index.php, procesar.php

Objetivo del proyecto

Aplicar el uso de arreglos unidimensionales paralelos en PHP para almacenar, procesar y analizar información de productos de una tienda en línea, calculando el total, promedio, producto más caro y más barato, y presentando los resultados en una tabla HTML estructurada.

Problema planteado

Una tienda en línea necesita un módulo para gestionar su inventario. El sistema debe permitir al usuario registrar al menos 5 productos con nombre y precio desde un formulario, y luego calcular estadísticas básicas del inventario usando funciones nativas de PHP.

Conceptos de POO aplicados

- Arreglos unidimensionales paralelos (\$productos[] y \$precios[]).
- Formularios HTML con validación básica (required, type=number).
- Procesamiento de datos con funciones nativas: array_sum(), max(), min().
- Uso de array_search() para encontrar el índice del producto máximo/mínimo.
- Separación de responsabilidades: index.php (formulario) y procesar.php (lógica y resultados).
- Presentación de resultados en tabla HTML semántica.

Explicación del funcionamiento

El proyecto se dividió en dos scripts:

- index.php: Contiene un formulario HTML que solicita 5 pares nombre-precio. Cada campo de precio usa type='number' con step='0.01' para validación básica en el navegador. Al enviar, los datos van por POST a procesar.php.
- procesar.php: Recibe los arrays \$_POST['productos'] y \$_POST['precios'], valida que no haya campos vacíos y los almacena en \$productos[] y \$precios[]. Luego calcula: precio total con array_sum(\$precios), promedio como total/count(\$precios), producto más caro usando max(\$precios) y array_search() para obtener su nombre, y el más barato con min(\$precios) igualmente. Los resultados se presentan en una tabla HTML con encabezados y filas de datos.

Capturas del proyecto



Resultados del Inventario

TiendaOnline — Análisis de productos registrados



TOTAL INVENTARIO

\$75.00

5 productos registrados



PRECIO PROMEDIO

\$15.00

Por producto



MÁS CARO

\$25.00

pc Thinkcentre



MÁS BARATO

\$5.00

Tablet hp

Lista de Productos

#	Producto	Precio	Estado
1	laptop asus	\$10.00	● Bajo promedio
2	Laptop Lenovo	\$15.00	● En promedio
3	Tablet hp ↓ Más barato	\$5.00	● Bajo promedio
4	Laptop Dell	\$20.00	● Sobre promedio
5	pc Thinkcentre ↑ Más caro	\$25.00	● Sobre promedio
Totales		\$75.00	Promedio: \$15.00

Captura de Productos

Instrucciones: Ingresa el nombre y precio de al menos 5 productos. Todos los campos son obligatorios y el precio debe ser un valor positivo mayor a cero.

1

Producto:

laptop asus

Precio (\$):

10.000

2

Producto:

Laptop Lenovo

Precio (\$):

15.000

3

Producto:

Tablet hp

Precio (\$):

5.000

4

Producto:

Laptop Dell

Precio (\$):

20.000

5

Producto:

pc Thinkcentre

Precio (\$):

25.000

Dificultades encontradas

- Entender cómo PHP recibe arrays desde formularios HTML con la notación `name='campo[]'`.
- Usar `array_search()` correctamente para obtener el índice del valor máximo/mínimo.
- Asegurar que los índices de `$productos` y `$precios` estén sincronizados tras recibirlos del formulario.

Soluciones aplicadas

- Investigar la documentación de PHP sobre arrays en formularios HTML y practicar con un ejemplo mínimo.
- Probar `array_search(max($arr), $arr)` en aislado antes de integrarlo al script principal.
- Verificar con `var_dump()` que ambos arrays tengan el mismo número de elementos antes de procesarlos.

Proyecto 4: Gestión de Bitácoras en Archivos de Texto (PHP)

Proyecto	Gestión de Bitácoras en Archivos de Texto
Materia	Programación Orientada a Objetos
Lenguaje	PHP
Tipo de actividad	Actividad de evaluación ABP
Archivos	index.php, bitacora.txt

Objetivo del proyecto

Desarrollar un módulo prototipo en PHP que permita registrar, almacenar y consultar actividades diarias en un archivo de texto plano, sin el uso de bases de datos, aplicando buenas prácticas de validación, manejo de archivos y presentación de datos.

Problema planteado

Una empresa de seguridad lleva registro manual de actividades en papel. El director desea digitalizar este proceso usando archivos de texto, ya que no cuentan con infraestructura de bases de datos. Se necesita un sistema web que permita registrar actividades con descripción, responsable y fecha, guardarlas de forma persistente y consultarlas ordenadamente.

Conceptos de POO aplicados

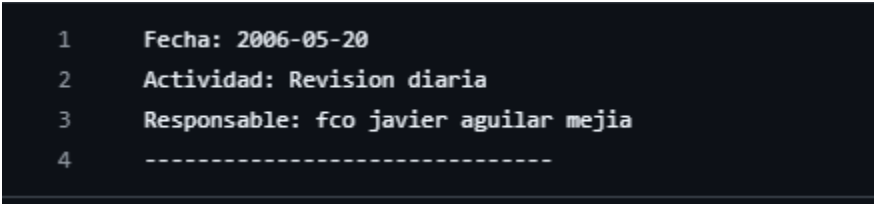
- Manipulación de archivos en PHP: `file_put_contents()` con `FILE_APPEND`, `file_get_contents()`.
- Validación de formularios: verificación de campos vacíos en PHP.
- Formato estructurado de datos en archivo de texto plano.
- Prevención de inyección de código con `htmlspecialchars()`.
- Presentación de datos con etiquetas HTML semánticas (`<pre>`, `<ol>`).
- Manejo de mensajes de éxito y error para retroalimentación al usuario.

Explicación del funcionamiento

El proyecto consiste en un único archivo `index.php` con dos bloques funcionales:

- Bloque de escritura: Un formulario HTML captura la descripción de la actividad, el responsable y la fecha (`input type='date'`). Al enviar el formulario, se valida que ningún campo esté vacío. Si pasa la validación, se formatea el registro como `'Fecha: YYYY-MM-DD\nActividad: texto\nResponsable: texto\n---'` y se escribe en `bitacora.txt` usando `file_put_contents()` con la bandera `FILE_APPEND` para no sobrescribir registros anteriores. Se muestra un mensaje de éxito verde.
- Bloque de lectura: Inmediatamente después del formulario, se verifica si `bitacora.txt` existe con `file_exists()`. Si existe, se lee su contenido con `file_get_contents()` y se muestra dentro de una etiqueta `<pre>` o se parsea línea por línea para mostrarlo en una lista `<ol>` ordenada.

Capturas del proyecto

A screenshot of a text file with a dark background and light-colored text. The text is organized into four numbered lines. Line 1 shows 'Fecha: 2006-05-20'. Line 2 shows 'Actividad: Revision diaria'. Line 3 shows 'Responsable: fco javier aguilar mejia'. Line 4 shows a series of dashes '-----'.

```
1    Fecha: 2006-05-20
2    Actividad: Revision diaria
3    Responsable: fco javier aguilar mejia
4    -----
```

Dificultades encontradas

- Recordar usar FILE_APPEND en file_put_contents() para no sobrescribir el archivo.
- Prevenir que usuarios ingresen caracteres especiales HTML que pudieran interpretarse como código.
- Gestionar el caso en que el archivo bitacora.txt no exista aún en la primera ejecución.

Soluciones aplicadas

- Consultar la documentación de file_put_contents() y comparar el comportamiento con y sin FILE_APPEND.
- Aplicar htmlspecialchars() a todos los datos antes de escribirlos en el archivo.
- Usar file_exists() antes de intentar leer el archivo para evitar warnings de PHP.

Proyecto 5: Sistema de Usuarios con Herencia en Python

Proyecto	Sistema de Usuarios con Herencia en Python
Materia	Programación Orientada a Objetos
Lenguaje	Python
Tipo de actividad	ABPj – Actividad de evaluación Corte 3
Archivos	usuario.py, admin.py, cliente.py, invitado.py, main.py

Objetivo del proyecto

Desarrollar un sistema orientado a objetos en Python utilizando herencia para modelar distintos tipos de usuarios (Admin, Cliente, Invitado), aplicando reutilización de atributos y métodos, polimorfismo y sobrescritura, además de documentar la solución mediante una infografía técnica.

Problema planteado

Una plataforma digital necesita controlar distintos tipos de usuarios con comportamientos y permisos distintos. Todos comparten información básica (nombre, email), pero cada tipo tiene acciones específicas al sistema. Se requiere modelar esta jerarquía usando herencia en Python.

Conceptos de POO aplicados

- Herencia en Python: clases derivadas que extienden una clase base.
- Constructor `__init__` y uso de `super()` para inicializar atributos heredados.
- Sobreescritura de métodos (override): `acceso_sistema()` diferente en cada clase.
- Polimorfismo: recorrer una lista heterogénea de objetos llamando al mismo método.
- Atributos propios de subclase: `nivel_acceso` en Admin, `puntos` en Cliente.
- Encapsulamiento y organización en múltiples archivos `.py`.
- Validación de email mediante expresiones regulares o lógica básica.

Explicación del funcionamiento

El sistema se divide en cinco archivos:

- `usuario.py` – Clase base con atributos `nombre` y `email`, métodos `mostrar_datos()` que imprime la información del usuario y `acceso_sistema()` que muestra un mensaje genérico de acceso.
- `admin.py` – Hereda de Usuario, agrega `nivel_acceso`, sobrescribe `acceso_sistema()` para mostrar los permisos de administrador.
- `cliente.py` – Hereda de Usuario, agrega `puntos`, sobrescribe `acceso_sistema()` para mostrar el saldo de puntos disponibles.
- `invitado.py` – Hereda de Usuario, sobrescribe `acceso_sistema()` indicando acceso limitado de solo lectura.

- main.py – Crea instancias de Admin, Cliente e Invitado, muestra datos de cada uno y demuestra polimorfismo recorriendo todos en una lista y llamando a acceso_sistema() en cada objeto.

Capturas del proyecto

```
MENÚ PRINCIPAL
1. Ver todos los usuarios
2. Saludar a todos
3. Agregar usuario
4. Salir

Elige una opción: 3

--- AGREGAR USUARIO ---
Tipo: 1) Admin 2) Cliente 3) Invitado
Selecciona tipo: 3
Nombre: fco
Email: gmail.com
Error: Email inválido: gmail.com

MENÚ PRINCIPAL
1. Ver todos los usuarios
2. Saludar a todos
3. Agregar usuario
4. Salir

Elige una opción: 1

=====
LISTA COMPLETA DE USUARIOS
=====

[1] Admin
Nombre : Carlos Mendoza
Email : carlos@itsl.edu.mx
Nivel de acceso: 5
[Carlos Mendoza] ✓ Acceso ADMINISTRADOR - Nivel 5.
Puede gestionar usuarios, configurar el sistema y ver reportes.
=====
```

```

[1] Admin
  Nombre : Carlos Mendoza
  Email  : carlos@itsl.edu.mx
  Nivel de acceso: 5
  [Carlos Mendoza] ✓ Acceso ADMINISTRADOR - Nivel 5.
  Puede gestionar usuarios, configurar el sistema y ver reportes.
-----

[2] Admin
  Nombre : Laura Ríos
  Email  : laura@itsl.edu.mx
  Nivel de acceso: 3
  [Laura Ríos] ✓ Acceso ADMINISTRADOR - Nivel 3.
  Puede gestionar usuarios, configurar el sistema y ver reportes.
-----

[3] Cliente
  Nombre : Ana Torres
  Email  : ana@gmail.com
  Puntos acumulados: 1200
  [Ana Torres] ✓ Acceso CLIENTE - 1200 puntos disponibles.
  Puede ver productos, realizar compras y consultar su historial.
-----

[4] Cliente
  Nombre : Luis Herrera
  Email  : luis@gmail.com
  Puntos acumulados: 350
  [Luis Herrera] ✓ Acceso CLIENTE - 350 puntos disponibles.
  Puede ver productos, realizar compras y consultar su historial.
-----

[5] Invitado
  Nombre : Visitante1
  Email  : visita1@temp.com
  [Visitante1] ⚠ Acceso INVITADO - Solo lectura.
  Solo puede ver contenido público. Regístrate para más funciones.

```

SALUDOS DEL SISTEMA

```

;Hola, Carlos Mendoza! Bienvenido al sistema.
;Hola, Laura Ríos! Bienvenido al sistema.
;Hola, Ana Torres! Bienvenido al sistema.
;Hola, Luis Herrera! Bienvenido al sistema.
Hola, Visitante1. Tienes acceso de invitado (solo lectura).
Hola, Visitante2. Tienes acceso de invitado (solo lectura).

```

```
[7] Admin
Nombre : Aguilar
Email : Aguilar@gmail.com
Nivel de acceso: 4
[Aguilar] ✓ Acceso ADMINISTRADOR - Nivel 4.
Puede gestionar usuarios, configurar el sistema y ver reportes.
-----
```

```
[9] Cliente
Nombre : pacote
Email : pacote@gmail.com
Puntos acumulados: 2000
[pacote] ✓ Acceso CLIENTE - 2000 puntos disponibles.
Puede ver productos, realizar compras y consultar su historial.
-----
```

Dificultades encontradas

- Adaptarse a la sintaxis de Python después de trabajar con PHP durante el semestre.
- Usar `super().__init__()` correctamente para no duplicar la inicialización de atributos heredados.
- Implementar validación de email sin librerías externas de forma robusta.
- Crear y estructurar la infografía técnica requerida como entregable adicional.

Soluciones aplicadas

- Estudiar un tutorial básico de Python orientado a objetos para identificar las diferencias sintácticas con PHP.
- Practicar `super().__init__()` con un ejemplo mínimo de dos clases antes de aplicarlo al proyecto completo.
- Implementar una validación de email simple verificando la presencia de '@' y '.' en el string.
- Usar Canva o Google Slides para crear la infografía con un template técnico y completar cada sección requerida.

Reflexión Final

¿Cuál proyecto fue más difícil?

El proyecto más difícil fue el Analizador de Consumo Energético en Servidores (Corte 2), ya que requería integrar varios conceptos simultáneamente: namespaces, encapsulamiento, manejo de excepciones y un algoritmo matemático (Regla del Trapecio) dentro de una arquitectura de proyecto real. Entender cómo conectar la clase con su namespace al index.php usando use fue el punto de mayor dificultad técnica.

¿Cuál proyecto fue más útil?

El proyecto más útil en términos de aplicación práctica fue el Sistema de Bitácoras en Archivos de Texto, ya que simula una necesidad real muy común en el entorno laboral: registrar eventos de forma persistente sin depender de una base de datos. Esta técnica es ampliamente utilizada en sistemas de logging, auditoría y diagnóstico en entornos de producción.

¿Qué habilidad mejoró más?

La habilidad que más mejoró fue el pensamiento orientado a objetos y la capacidad de abstraer un problema del mundo real en una clase con atributos y métodos bien definidos. Al inicio del semestre, la tendencia era escribir todo el código en un solo archivo de manera procedural. Al finalizar, se logró dividir responsabilidades, aplicar encapsulamiento de forma natural y diseñar clases reutilizables.

¿Qué aprendiste sobre Programación Orientada a Objetos?

Aprendí que la POO no es solo una forma de escribir código, sino una filosofía de diseño de software. Los pilares del paradigma —encapsulamiento, herencia, polimorfismo y abstracción— no son conceptos aislados, sino herramientas complementarias que, usadas correctamente, permiten construir sistemas escalables, mantenibles y comprensibles. También aprendí que la POO facilita enormemente el trabajo en equipo, ya que cada clase puede ser desarrollada y probada de forma independiente.

¿Qué aprendiste sobre trabajo organizado?

Aprendí que la organización del código es tan importante como el código mismo. Separar las clases en archivos individuales, usar namespaces para evitar colisiones de nombres, documentar cada método y mantener una estructura de carpetas coherente son prácticas que hacen la diferencia entre un proyecto amateur y uno profesional. El trabajo organizado no solo facilita el desarrollo actual, sino que facilita el mantenimiento futuro por cualquier colaborador.

Conclusiones

Crecimiento técnico

A lo largo del semestre se desarrolló una base sólida en el paradigma de Programación Orientada a Objetos, partiendo desde la definición de clases simples con propiedades tipadas hasta la implementación de jerarquías de herencia en dos lenguajes distintos (PHP y Python). Cada proyecto añadió una capa de complejidad: del modelado de datos al encapsulamiento, de los métodos a las excepciones, de la herencia al polimorfismo. Este crecimiento progresivo permitió internalizar los conceptos de forma orgánica, relacionando la teoría con problemas reales.

Importancia del desarrollo de proyectos

El aprendizaje basado en proyectos demostró ser significativamente más efectivo que el estudio teórico aislado. Al enfrentarse a un problema concreto —calcular el consumo energético de un servidor, gestionar usuarios de una plataforma, mantener una bitácora digital— los conceptos abstractos de la POO tomaron sentido práctico. El desarrollo de proyectos obliga a tomar decisiones de diseño, depurar errores reales y documentar el trabajo, habilidades que definen a un ingeniero informático competente.

Importancia de la documentación y GitHub

La documentación es la memoria del software. Sin ella, un proyecto que funciona hoy puede ser incomprensible en seis meses. A través de este portafolio se practica precisamente esa cultura de documentación: explicar el objetivo, el problema, las decisiones técnicas y las lecciones aprendidas de cada proyecto. GitHub complementa esta práctica al ofrecer un historial versionado del código, permitiendo recuperar versiones anteriores, comparar cambios y demostrar la evolución del trabajo. En el entorno laboral, un desarrollador que sabe documentar y usar Git es un activo más valioso que uno que solo sabe programar.

Jesús Salas Marín

Ingeniería Informática – Programación Orientada a Objetos

Instituto Tecnológico Superior de Lerdo | Junio 2026